

Guía de Desarrollo: Marca Personal FP (Parte 2)

Continuación de los Bloques 1 y 2. Misma estructura en cada apartado:

- ¿Qué es? → Teoría mínima.
- Ejemplo genérico → Código de muestra (tienda de **Videojuegos**). **NO lo copies al proyecto.**
- Reto del proyecto → Lo que TÚ programas. Te doy comandos y guía, **no el código.**

Requisito previo: Antes de empezar este bloque debes tener la base de datos creada y poblada (Bloque 2 terminado).

BLOQUE 3: Capa de Dominio (Modelos y Servicios)

3.1. Modelos Eloquent

¿Qué es y para qué sirve?

- Un **modelo Eloquent** representa **una tabla** y te deja trabajar con ella como si fueran **objetos PHP**, sin escribir SQL.
- **\$fillable** lista los campos que se pueden rellenar de forma masiva (**create()**, **update()**). Protege contra el *mass assignment*.
- Los **métodos de relación** conectan modelos entre sí:
 - **belongsToMany** → "pertenece a uno" (lado de la FK).
 - **hasMany** → "tiene muchos" (lado opuesto).
 - **belongsToMany** → "muchos a muchos" (usa tabla pivot).

Ejemplo genérico (tienda de Videojuegos)

Shell

```
php artisan make:model Videojuego
```

PHP

```
class Videojuego extends Model
{
    protected $fillable = [
        'titulo', 'descripcion', 'precio', 'stock',
        'plataforma', 'disponible', 'slug',
    ];

    // Un videojuego pertenece a una empresa (1:N)
    public function empresa()
    {
        return $this->belongsTo(Empresa::class);
    }

    // Un videojuego tiene muchas categorías (N:M)
    public function categorias()
    {
        return $this->belongsToMany(Categoria::class);
    }
}
```

Pista: El nombre del método de relación marca el tipo de acceso: `$juego->empresa` (objeto único) vs `$juego->categorias` (colección).

Reto del proyecto

1. **Crea cada modelo** (si no existe ya):

Shell

```
php artisan make:model NombreModelo
```

- 2.

Define `$fillable` en cada modelo con los campos de su tabla (los de las migraciones del Bloque 2).

3. **Implementa las relaciones** según esta tabla:

Modelo	Relación a programar	Método Eloquent
--------	----------------------	-----------------

FamiliaProfesional	Tiene muchos ciclos	hasMany(CicloFormativo::class)
CicloFormativo	Pertenece a una familia	belongsTo(FamiliaProfesional::class)
CicloFormativo	Tiene muchos estudiantes	belongsToMany(Estudiante::class)
Estudiante	Pertenece a un user	belongsTo(User::class)
Estudiante	Cursa muchos ciclos	belongsToMany(CicloFormativo::class)
Estudiante	Participa en muchos proyectos	belongsToMany(Proyecto::class)
Docente	Pertenece a un user	belongsTo(User::class)
Docente	Supervisa muchos proyectos	belongsToMany(Proyecto::class)
Proyecto	Tiene muchos estudiantes	belongsToMany(Estudiante::class)
Proyecto	Tiene muchos docentes	belongsToMany(Docente::class)

Pistas importantes:

- Si el **nombre de la tabla pivot** no sigue la convención de Laravel, pásalo como 2º argumento: `belongsToMany(Estudiante::class, 'ciclo_formativo_estudiante')`.
- Comprueba las relaciones en `php artisan tinker` (ej. `Estudiante::first()->proyectos`).

Objetivo: Acceder a datos relacionados sin errores desde Tinker.

3.2. Capa de Servicios (Services)

¿Qué es y para qué sirve?

- Un **Service** es una clase que **contiene la lógica de negocio**: crear, buscar, actualizar, borrar, cálculos, reglas.

- **Objetivo:** que el **controlador quede limpio**. El controlador recibe la petición y delega el trabajo al servicio.
- Ventajas: código **reutilizable**, **fácil de testear** y **fácil de mantener**.

Ejemplo genérico (tienda de Videojuegos)

Se crea a mano en `app/Services/VideojuegoService.php`:

```
PHP
namespace App\Services;

use App\Models\Videojuego;

class VideojuegoService
{
    public function listar()
    {
        return Videojuego::with('categorias')->paginate(10);
    }

    public function crear(array $datos): Videojuego
    {
        return Videojuego::create($datos);
    }

    public function actualizar(Videojuego $videojuego, array $datos): Videojuego
    {
        $videojuego->update($datos);
        return $videojuego;
    }

    public function eliminar(Videojuego $videojuego): void
    {
        $videojuego->delete();
    }
}
```

Pista: Los servicios **no** se generan con Artisan; se crean como clase PHP normal dentro de la carpeta `app/Services`.

Reto del proyecto

1. Crea la carpeta `app/Services`.
2. Crea un servicio por entidad principal (clase PHP, sin Artisan):

Servicio	Métodos mínimos a implementar
<code>ProyectoService</code>	<code>listar()</code> , <code>crear()</code> , <code>actualizar()</code> , <code>eliminar()</code>
<code>EstudianteService</code>	<code>listar()</code> , <code>crear()</code> , <code>actualizar()</code> , <code>eliminar()</code>
<code>CicloFormativoService</code>	<code>listar()</code> , <code>crear()</code> , <code>actualizar()</code> , <code>eliminar()</code>

3. Mete la lógica de negocio aquí (no en el controlador). Ejemplos de lógica propia:
 - Vincular un proyecto con sus estudiantes/docentes (`sync()` / `attach()`).
 - Cargar relaciones con `with()` para evitar el problema N+1.

Objetivo: Toda la lógica de datos vive en `app/Services`, lista para inyectarse en el Bloque 4.

BLOQUE 4: Controladores y Rutas

4.1. Definición de Rutas (`web.php`)

¿Qué es y para qué sirve?

- En `routes/web.php` defines qué URL ejecuta qué método de qué controlador.
- `Route::resource` crea de golpe las 7 rutas CRUD (index, create, store, show, edit, update, destroy).
- Grupos y middleware aplican reglas comunes a varias rutas. El middleware `auth` obliga a estar logueado.

Ejemplo genérico (tienda de Videojuegos)

```
PHP
use App\Http\Controllers\VideojuegoController;

// Ruta pública
```

```
Route::get('/', [VideojuegoController::class,
'index'])->name('home');

// Rutas protegidas: solo usuarios autenticados
Route::middleware('auth')->group(function () {
    Route::resource('videojuegos', VideojuegoController::class);
});
```

Ver todas las rutas generadas:

```
Shell
php artisan route:list
```

Reto del proyecto

1. Define las rutas **resource** de tus entidades dentro de un **grupo con middleware auth**.

Recurso	Ruta resource
Proyectos	<code>Route::resource('proyectos', ProyectoController::class)</code>
Estudiantes	<code>Route::resource('estudiantes', EstudianteController::class)</code>
Ciclos formativos	<code>Route::resource('ciclos', CicloFormativoController::class)</code>

2.

Decide qué es público y qué es privado:

- **Público:** ver listados / detalle (ej. `index`, `show`).
- **Privado (auth):** crear, editar, borrar (ej. `create`, `store`, `edit`, `update`, `destroy`).
- Pista: puedes separar rutas con `->only([...])` y `->except([...])`.

3. **Verifica:**

```
Shell
php artisan route:list
```

Objetivo: Las rutas de edición/borrado solo accesibles logueado; los listados visibles para todos.

4.2. Controladores

¿Qué es y para qué sirve?

- Un **controlador** recibe la **petición**, llama al **servicio** y devuelve una **vista** o una **redirección**.
- **Inyección de dependencias:** Laravel pasa automáticamente el servicio por el **constructor**. No haces **new**.
- Un **controlador de recurso** trae los 7 métodos CRUD listos para rellenar.

Ejemplo genérico (tienda de Videojuegos)

Shell

```
php artisan make:controller VideojuegoController --resource
```

PHP

```
class VideojuegoController extends Controller
{
    public function __construct(
        private VideojuegoService $service
    ) {}

    public function index()
    {
        $videojuegos = $this->service->listar();
        return view('videojuegos.index', compact('videojuegos'));
    }

    public function store(Request $request)
    {
        $datos = $request->validate([
            'titulo' => 'required|string|max:255',
            'precio' => 'required|numeric',
        ]);

        $this->service->crear($datos);
        return redirect()->route('videojuegos.index');
```

```
}  
}
```

Pista: El controlador **no contiene lógica de negocio**: solo valida, delega en el servicio y responde.

Reto del proyecto

1. Crea un controlador de recurso por entidad:

```
Shell  
php artisan make:controller NombreController --resource
```

Controlador	Servicio que inyecta
ProyectoController	ProyectoService
EstudianteController	EstudianteService
CicloFormativoController	CicloFormativoService

2.
Inyecta el servicio por el constructor (como en el ejemplo).
3. **Rellena los métodos CRUD** delegando en el servicio:

Método	Qué debe hacer
index	Pedir el listado al servicio y enviarlo a la vista
create	Mostrar el formulario de creación
store	Validar y llamar a <code>crear()</code> del servicio
show	Mostrar el detalle de un registro
edit	Mostrar el formulario de edición
update	Validar y llamar a <code>actualizar()</code> del servicio

`destro` Llamar a `eliminar()` y redirigir
y

Pistas importantes:

- **Valida siempre** la entrada con `$request->validate([...])` antes de tocar la BD.
- Usa **Route Model Binding**: tipa el parámetro (ej. `Proyecto $proyecto`) y Laravel busca el registro solo.
- Tras crear/editar/borrar, **redirige** con `redirect()->route(...)`.

Objetivo: CRUD completo funcionando vía navegador, con el controlador limpio y la lógica en los servicios.

Has terminado Bloques 3 y 4. Ya tienes modelos relacionados, lógica en servicios, rutas protegidas y controladores CRUD. El siguiente paso (Bloque 5) será **construir las vistas Blade** (layouts, directivas y formularios) para que el usuario interactúe con todo esto.